

تقرير شرح مشروع مساعد سرطان القولون والمستقيم

شرح فني منظم لصفحة التحليل، خط سير RAG، المكتبات، الموديلات، الملفات، والفانكشنز الأساسية

البند	القيمة الحالية
الموضوع	Colorectal cancer guideline assistant
المصادر	NICE NG12 من colorectal جزء + NICE NG151
عدد الصفحات المستخرجة من NG151	51
عدد chunks النهائي	95
Embedding model	intfloat/multilingual-e5-base
Vector DB	Chroma
Production Top-k	5
Evaluation questions	130 total / 122 in-scope / 8 out-of-scope

1. ملخص سريع للمشروع

ثم يرجع إجابة مختصرة مع مصدر. الهدف الأساسي أن الإجابة لا تعتمد على معرفة عامة من الموديل، بل على النصوص المسترجعة من الدليل. RAG ثنائي اللغة عربي وإنجليزي. المستخدم يسأل سؤال عن سرطان القولون والمستقيم، والسيستم يبحث في guideline رسمي من NICE، المشروع عبارة عن Assistant

- وتحويل لنص منظم parsing وcleaning عملنا PDF، اخترنا المصدر الرسمي، نزلنا: Day 1
- evaluation. وبنينا Chroma، خزناها في embeddings، عملنا chunks، قسمنا النص إلى: Day 2
- عشان يحول الدليل المسترجع إلى إجابة مفهومة ومختصرة generation model أضفنا: Day 3
- خاصة تعرض جودة الاسترجاع Analysis وصفحة Chat UI، احترافي Frontend بنينا: Day 4

العام: architecture

```
NICE PDFs -> ingest.py -> clean text -> chunks -> embeddings -> Chroma
Question -> query.py -> Top-5 evidence -> generate.py -> cited answer -> Next.js chat
Admin -> /analysis -> evaluate.py -> metrics and question-level evidence
```

2. صفحة Analysis بتعرض إيه وكل رقم جاي منين؟

عند فتح الصفحة، تعمل request على /api/analysis/evaluate/، والAPI يشغل evaluate.py --top-k 5 --json، ثم يعرض النتائج. في Next.js داخل web/components/dashboard.tsx. الصفحة Private ومحمية بكلمة سر من ANALYSIS_PASSWORD. صفحة Analysis موجودة

جاي منين في الكود	معناه	العنصر في الصفحة
evaluate.py -> build_summary -> found_rate	هل الدليل المتوقع ظهر داخل أول 5 نتائج.	Found Rate@5
ثم evaluate.py -> precision_at_k mean_precision_at_k	متوسط نسبة النتائج الصحيحة من الخمسة المسترجعة.	Mean Precision@5
evaluate.py -> average_precision_at_k ثم map_at_k	الأدلة الصحيحة، كلما الدليل الصحيح يظهر أبكر الرقم يزيد. يقيس جودة ترتيب	MAP@5
ثم mrr evaluate.py -> reciprocal_rank	ترتيب أول نتيجة صحيحة. لو أول نتيجة صح يبقى 1.0. يقيس	MRR
evaluate.py -> build_summary -> ar_found_count / en_found_count	تغطية كل لغة منفصلة.	Arabic / English
evaluate.py -> rank_1_count, top_3_count, top_5_count, missed_count	توزيع مكان أول دليل صحيح.	Rank 1 / Top 3 / Top 5 / Missed
ثم dashboard.tsx -> rows evaluate.py -> QuestionTable	أول chunk، الصفحة، score، والتوصية المتوقعة. كل سؤال، النتيجة،	Question-level evidence
dashboard.tsx -> runEvaluation + /api/analysis/evaluate/route.ts	تجربة k مختلفة مؤقتًا بدون تغيير k الأساسي.	Temporary Top-k evaluation

3. النتائج الحالية ولماذا هي منطقية

التفسير	Current value	Metric
122/122 أسئلة داخل النطاق وجدنا لها الدليل المتوقع	100.00%	Found Rate@5
من Found Rate لأننا نرجع 5 chunks والسؤال غالبا له chunk صحيح واحد أو اثنين طبيعى يكون أقل	25.74%	Mean Precision@5
بدل أن الدليل الصحيح غالبا متقدم في الترتيب	82.63%	MAP@5
أول دليل صحيح غالبا يظهر قريب جدا من المركز الأول	86.15%	MRR
كل الأسئلة العربية داخل النطاق نجحت	61/61	Arabic
كل الأسئلة الإنجليزية داخل النطاق نجحت	61/61	English

الاستنتاج	Precision	Found Rate	Top-k
دقيق جدا لكنه ممكن يفوت الدليل لو لم يكن أول نتيجة	77.05%	77.05%	k=1
توازن جيد	39.62%	95.90%	k=3
أفضل تغطية من k=3	30.94%	97.54%	k=4
اختيارنا الأساسي لأنه يغطي كل الأدلة المتوقعة	25.74%	100.00%	k=5

4. شرح RAG Pipeline خطوة بخطوة

- Parsing: مع الاحتفاظ بالنص والجدول قدر الإمكان pymupdf4llm باستخدام Markdown وتحويله إلى PDF قراءة
- Cleaning: المسافات الزائدة، وتنسيقات غير مفيدة، Markdown نجوم، tags مثل PDF الناتج من noise إزالة
- Chunking: 1.6.1، ومع الجداول يتم الحفاظ على NICE جديد عند أرقام توصيات chunk أصغر مرتبطة بالبنية. يبدأ chunks تقسيم النص إلى قدر الإمكان rows مثل
- Embedding: SentenceTransformer model: intfloat/multilingual-e5-base باستخدام vector chunk تحويل كل
- Vector store: مثل الصفحة، اسم المستند، نوع المحتوى، ورابط المصدر metadata مع Chroma داخل vectors تخزين
- Retrieval: cosine similarity باستخدام chunks يرجع أقرب Chroma للسؤال، ثم embedding عند السؤال، يتم
- Generation: لإنتاج إجابة قصيرة بمصدر Groq/Qwen المسترجعة إلى chunks يرسل السؤال والـ generate.py
- Safety: السيستم يرفض بأمان، citation format أو الموديل خرج عن threshold أقل من score لو مفيش دليل كافي، أو

5. نوع chunking المستخدم

لأننا لا نقسم بناء على embedding similarity بين الجمل. وليس Fixed-size فقط، لأننا لا نقص كل 450 token بشكل أعمى. Structure-aware. يعني هو خليط بين document-based chunking و recursive chunking. ليس Semantic chunking، المستخدم هنا هو Recursive Chunking with token-aware overlap

- Document-aware: لأننا نحترم أرقام التوصيات، الصفحات، الجداول، والعناوين
- Recursive: مثل فقرات ثم سطور ثم جمل ثم كلمات separators يجرب LangChain RecursiveCharacterTextSplitter لأن
- Token-aware: وليس بعدد الحروف فقط multilingual-e5-base لأن الحجم محسوب بتوكيزر
- Overlap = 80 tokens: والثاني chunk لتقليل فقدان السياق بين
- Chunk size = 450 tokens: metadata. metatrunkنا مساحة لل token له حد قريب من 512 model e5-base لأن

6. المكتبات ولماذا استخدمناها

ليه دي؟	استخدامها	Library
PDF بشكل أوضح من raw PyMuPDF وتحافظ على الجداول نسبياً مناسبة لاستخراج نصوص	PDF parsing إلى Markdown/page chunks	pymupdf4llm
مطابقة للنمط المطلوب في الهاكاوثون ومفهومة وسهلة الشرح	RecursiveCharacterTextSplitter	langchain-text-splitters
مدفوع API ويدعم عربي/إنجليزي بدون Open-source	عمل embeddings تحميل intfloat/multilingual-e5-base	sentence-transformers
سهل، سريع للتجارب، ولا يحتاج سيرفر منفصل	محلي Vector database	chromadb
محلي LLM سريع ومناسب بدل تشغيل API cloud	ربط Groq generation model	langchain-groq
يحمي المفاتيح من وضعها داخل الكود	تحميل مفاتيح API من env.	python-dotenv

Library	استخدامها	ليه دي؟
arabic-reshaper + python-bidi	إصلاح عرض العربي في Terminal/PDF	لأن بعض البيانات تعرض العربي معكوس أو حروف منفصلة
Next.js	Frontend و API routes	backend routes مناسب لواجهة ويب حديثة و Framework في نفس المشروع
TypeScript	واضحة وأخطاء أقل types	يساعد في شرح data structures وتقليل runtime bugs
Tailwind CSS	تصميم سريع ومتناسق	مناسب لبناء UI احترافي بدون CSS طويل
Recharts	رسوم بيانية في Analysis	سهل لعرض مقارنة k والmetrics
lucide-react	Icons	خفيفة وواضحة ومناسبة للواجهة

7. الموديلات المستخدمة

السبب	مكان الاستخدام	Model
عربي وإنجليزي والداتا إنجليزية. أفضل من Embedding model في تجاربنا small متعدد اللغات. مناسب لأن الأسئلة	ingest.py, query.py, evaluate.py	intfloat/multilingual-e5-base
evidence يستخدم فقط لصياغة الإجابة من Generation model المسترجع. ليس مصدر المعرفة الأساسي	generate.py	qwen/qwen3.6-27b على Groq

الـ Generation model لا يبحث في الداتا ولا يخزن معلومات؛ هو فقط يحول chunks المسترجعة إلى إجابة مفهومة بشرط الالتزام بالمصدر. مهم في الشرح: الـ Embedding model هو المسؤول عن البحث الدلالي.

8. مصادر الداتا

- المصدر الأساسي لعلاج وإدارة سرطان القولون والمستقيم: NICE NG151.
- والإحالة. لم ندخل باقي السرطانات، FIT، فقط لأنه يحتوي أعراض الاشتباه colorectal أضفنا منه جزء: NICE NG12.
- الأصلية PDF يحتوي ملفات: data/raw.
- parsing بعد pages و Markdown يحتوي: data/processed.
- metadata النهائية مع chunks يحتوي: data/chunks/chunks.jsonl.
- المحلية vectors قاعدة: data/vector_store/chroma.
- أسئلة الاختبار ونتائج التقييم: data/evaluation.

9. شرح الملفات الأساسية في Python

config.py

ملف الإعدادات المركزي. أي path أو model name أو chunk setting أو top-k يتغير من هنا بدل ما يتكرر في ملفات كثيرة.

Item	وظيفته
PDF_PATH / NG12_PDF_PATH	أماكن ملفات NICE PDF
MARKDOWN_PATH / PAGES_PATH / CHUNKS_PATH	أماكن مخرجات parsing و chunking
CHROMA_PATH / COLLECTION_NAME	مكان واسم قاعدة Chroma
CHUNK_SIZE / CHUNK_OVERLAP	حجم القطعة والتداخل
PAGES_TO_INDEX	الصفحات المفيدة فقط من NG151
EMBEDDING_MODEL	intfloat/multilingual-e5-base
TOP_K	عدد النتائج الأساسية = 5

Item	وظيفته
MIN_RETRIEVAL_SCORE	أقل score مقبول قبل الرفض الآمن

ingest.py

ده ملف بناء قاعدة المعرفة. بنشغله لما PDF يتغير أو لما نحتاج نعيد بناء chunks/vector store.

Function	بتعمل إيه؟
clean_markdown	تنظيف النص من HTML tags: noise، نجوم Markdown، مسافات كثيرة، semicolons من الجداول.
load_pdf	يقرأ PDF NG151 صفحة صفحة، يستخرج headings، يحفظ guideline.md و pages.json.
split_text	يقسم النص الكبير باستخدام LangChain Recursive splitter مع tokenizer حقيقي.
split_page	يبدأ chunk جديد عند كل recommendation number مثل 1.3.1.
content_type_for_page	يحدد نوع الصفحة: overview, recommendation, glossary, rationale.
table_id_for	يعطي الجداول id ثابت لو table امتدت على أكثر من chunk.
is_heading_only	يتجاهل heading fragment صغير مفهوش evidence.
is_cross_page_continuation	يعرف هل chunk مكمل من صفحة سابقة.
merge_cross_page_continuations	يدمج الأجزاء المقطوعة بين الصفحات.
page_label	يرجع page أو page range للاستشهاد.
token_count	يحسب tokens بنفس tokenizer بتاع e5.
split_large_record	يقسم chunk كبير مع الحفاظ على table rows قدر الإمكان.
chunk_pages	ينتج chunks النهائية مع metadata و text_for_embedding.
build_index	يعمل embeddings ويخزنها في Chroma.
write_jsonl	يحفظ records كسطر JSON لكل object.
main	يشغل pipeline كامل: قراءة، chunking، إضافة Chroma، embeddings NG12.

supplementary_sources.py

ملف مصدر إضافي. وظيفته استخراج جزء colorectal فقط من NICE NG12، لأن NG151 يحيل إلى NG12 في موضوع الاشتباه والإحالة.

ده

Function	بتعمل إيه؟
recommendation	تقص recommendation معينة من نص الصفحة بين start و end.
make_chunk	تحول recommendation من NG12 إلى chunk بنفس شكل chunks الأساسية.
load_ng12_colorectal_chunks	تقرأ صفحات NG12 المطلوبة وتستخرج توصيات 1.3.1 إلى 1.3.5 فقط.

الهدف منها تحسين فهم أسئلة المستخدم خصوصاً العربي والأسئلة المختصرة مثل: أعراض المرحلة الأولى، عندي سرطان أعمل إيه، أو انتشار للرئة. ده طبقة صغيرة قبل البحث.

وظيفته	Function / variable
قاموس intents: symptoms_referral, follow_up, liver_metastases, MSI/MMR كل intent له description و cues.	INTENTS
كلمات تثبت أن السؤال عن colorectal cancer.	DOMAIN_CUES
يوحد العربي: أ/إ/آ إلى ا، يشيل التشكيل، ينظف punctuation.	normalize_question
لو السؤال قصير ومث كاتب نوع السرطان، يضيف سياق سرطان القولون والمستقيم.	add_domain_context
يحدد intent إما بالكلمات الواضحة أو similarity بين السؤال ووصف intents.	understand_question
حساب lexical overlap بسيط يستخدم في reranking.	keyword_score

query.py

ده ملف retrieval الحقيقي. يستخدم embedding model و Chroma ويرجع أفضل chunks للسؤال.

بتعمل إيه؟	Function
يلتقط أسئلة معلومات المريض وفريق الرعاية ويحولها لقسم 1.2.	is_patient_information_question
يلتقط أسئلة early rectal cancer وجدول الاختيارات.	is_early_rectal_treatment_question
يلتقط أسئلة obstruction/stent.	is_bowel_obstruction_stent_question
يلتقط أسئلة انتشار الكبد.	is_liver_metastases_question
يفرق بين resectable و unresectable لتجنب خطأ قابل/غير قابل للاستئصال.	is_resectable_rectal_surgery_question
يلتقط أسئلة نقائل الرئة.	is_lung_metastases_question
يلتقط أسئلة الصفاق/peritoneum.	is_peritoneal_metastases_question
يلتقط أسئلة MSI/MMR والعلاج المناعي.	is_msi_mmr_immunotherapy_question
يزود السؤال بجملة guideline wording لتحسين retrieval دون API خارجي.	expand_question
يصحح عنوان القسم لو chunk بدأ بتوصية من قسم مختلف بسبب حدود الصفحة.	correct_section_title
يحمل embedding model مرة واحدة باستخدام lru_cache.	get_embedding_model
يفتح Chroma collection مرة واحدة.	get_collection
يعمل embeddings لعدة formulations، يبحث في Chroma، يزيل duplicates، ويعمل reranking. يفهم السؤال،	search

Function	بتعمل إيه؟
print_results / main	من الترمينال retrieval لاختبار CLI.

generate.py

النهائية. لا يبني knowledge base. هو يأخذ Top-k evidence من query.py، ثم يطلب من Groq/Qwen صياغة إجابة مقيدة بالمصدر. ده ملف الإجابة

Function / constant	وظيفته
SYSTEM_PROMPT	قواعد صارمة: أجب فقط من passages، لا تخمن، ارفض لو مفيش دليل مباشر.
is_arabic	يعرف لغة السؤال من وجود حروف عربية.
terminal_display	يعالج عرض العربي في terminal فقط.
output_instructions	يفرض format ثابت: التوصية، النص الداعم، المصدر.
citation_for	يبني citation عربي/إنجليزي من metadata.
build_context	يجمع passages المسموح للموديل استخدامها.
refusal_output	رسالة رفض أمانة لو لا توجد أدلة.
add_disclaimer	يضيف تنبيه طبي بأن الإجابة لا تغني عن الطبيب.
quota_output	رسالة واضحة لو Groq free limit خلص.
asks_for_stage_symptoms / stage_symptoms_output	يعالج سؤال أعراض مرحلة معينة بدون اختراع staging غير موجود في الدليل.
is_valid_answer	يتأكد من format الإجابة وأن citation ضمن المصادر المسموحة.
repair_answer_citation	يصلح citation لو الموديل كتب مصدر قريب لكن ليس مطابقا لـ metadata.
claims_are_supported	غير مدعومة claims إضافي بمنع إجابة فيها guard.
get_generation_model	يحمل Groq model مرة واحدة.
generate	الدالة الرئيسية: retrieve ثم generate أو refuse.
interactive_mode / main	تشغيل من الترمينال بسؤال واحد أو interactive.

evaluate.py

ده ملف قياس جودة الاسترجاع. لا يستخدم generation model، لأنه بقيس هل retrieval جاب evidence الصحيح قبل مرحلة الإجابة.

Function	بتعمل إيه؟
load_questions	يقرأ test_questions.csv.
expected_recommendations	يستخرج أرقام التوصيات ويتعامل مع ranges مثل 1.5.14-1.5.17.
contains_recommendation	chunk يتأكد من وجود رقم توصية كامل داخل Regex.
page_numbers	يفك page range مثل 10-12.

Function	بتعمل إيه؟
is_relevant	يحدد هل chunk مطابق للـ expected evidence، ويميز NG12 عن NG151.
average_precision_at_k	يحسب AP@k.
evaluate	يعمل embeddings لكل الأسئلة batch ثم Chroma query ثم rows تفصيلية.
build_summary	يحسب Found Rate, Precision, MAP, MRR, Arabic/English split.
build_json_report	يجهز JSON للصفحة.
print_summary	يطبع summary في الترمينال.
save_reports	يحفظ evaluation_results.csv و evaluation_summary.json.
main	CLI: top-k و json mode.

web_chat_bridge.py

كل request يحمل embedding model من الأول، Next.js يشغل process طويل العمر، يرسل له JSON line، ويرجع JSON line. ده جسر بين Next.js و Python. بدل

Function	بتعمل إيه؟
friendly_error	رسالة خطأ بسيطة حسب لغة السؤال.
main	يقرأ stdin، يعمل warmup، يشغل search/generate، ويطبع JSON response.

10. شرح ملفات الواجهة Frontend

web/app/page.tsx

الصفحة الرئيسية. تستدعي AssistantApp فقط. فصلناها عشان app router في Next.js يبقى بسيط.

web/app/layout.tsx

globals.css العنوان والوصف، ويستورد: metadata للموقع. يحدد Root layout.

web/components/assistant-app.tsx

GET /api/chat عبر RAG العام للشات: الهيدر، اسم الموقع، تبديل اللغة عربي/إنجليزي، وحالة جاهزية Shell.

web/components/chat-panel.tsx

في state، ترسل السؤال إلى /api/chat، تعرض loading dots، وتفصل الإجابة إلى recommendation/source/disclaimer. واجهة الشات. تحفظ الرسائل

وظيفتها	Function
تقسم إجابة generate.py إلى recommendation, excerpt, citation, disclaimer.	parseAnswer
يبنى رابط NICE مع anchor/text fragment لمحاولة فتح مكان المعلومة.	sourceUrl
يعرض الإجابة كمحادثة، ويخفي النص الداعم والمصدر داخل زر source NICE.	AssistantAnswer
الشات بالكامل: input, messages, loading, errors, submit.	ChatPanel

web/components/dashboard.tsx

صفحة التحليل الخاصة. تعرض metrics و charts و table تفصيلي. كما تسمح بتجربة k مؤقتا دون تغيير TOP_K الحقيقي.

وظيفته	Part
تعريف شكل البيانات القادمة من evaluate.py.	Summary types
أرقام احتياطية لو API لم يعمل.	fallbackSummary
يرسل POST إلى /api/analysis/evaluate.	runEvaluation
كروت Found Rate و Precision و MAP و MRR.	MetricCard
أرقام Rank 1 / Top 3 / Top 5 / Missed.	RankBox
نسب Overall/Arabic/English.	ProgressLine
نتيجة k المؤقت.	ExperimentResult
جدول كل سؤال وحالته وال chunk الأعلى.	QuestionTable

web/components/analysis-login.tsx

صفحة تسجيل دخول بسيطة للأناليزس. تطلب password وترسلها إلى /api/analysis/login/.

web/app/api/chat/route.ts

JSON response يرسل السؤال، وينتظر، warmup يعمل، سكيلد process web_chat_bridge.py للشات. يشغل API route. يعلق request لمنع timeout يستخدم.

web/app/api/analysis/evaluate/route.ts

لواجهة JSON ويرجع evaluate.py --top-k N --json ثم يشغل، session cookie خاص بالتحليل. يتحقق من API route.

web/app/api/analysis/login/route.ts و logout/route.ts

cookie يحذف logout، آمن cookie ويضع password يتحقق من login.

web/lib/analysis-auth.ts

يستخدم validSession، timingSafeEqual يتحقق من cookie، وanalysisConfigured يتأكد أن password موجود وطوله مناسب. منطق حماية صفحة التحليل: validPassword.

web/lib/data.ts

أرقام fallback و charts الثابتة: overview, topKData, modelData, chunkData. الصفحة تستخدمها كاحتياطي أو للمقارنات الثابتة.

11. لماذا استخدمنا هذه الاختيارات وليس غيرها؟

البديل ولماذا لم نختره الآن	لماذا؟	الاختيار
Chroma إضافية؛ metadata قوي لكنه يحتاج إدارة FAISS أبسط للعرض	محلي وسهل ومناسب للديمو والهاكاثون.	Chroma
مكلفة أو لها API embeddings أسرع لكنه أقل جودة؛ small limits.	يدعم العربي والإنجليزي ونتائجه أفضل من small.	multilingual-e5-base
قبل التقسيم embeddings أعقد ويحتاج Semantic chunking إضافية	مطلوب/مشهور وسهل الشرح.	LangChain splitter
لأنه لا يريد تشغيل محلي local LLM؛ limit؛ كان فيه Gemini المستخدم رفضه	محلي LLM ولا يحتاج تشغيل Cloud API	Groq/Qwen generation
يمكن لكن يزيد التعقيد في الشرح FastAPI منفصل Backend	في مشروع واحد Frontend و Backend bridge	Next.js API routes
وسيقا زائد noise أكبر يزيد k أدق لكنه يفوت أدلة؛ k=1	حقق Found Rate %100 على الأسئلة داخل النطاق.	k=5

12. كيف تشرح الـ metrics في المناقشة؟

- معناه أن الدليل الصحيح موجود ضمن أول 5 نتائج RAG. لنا في metric أهم Found Rate@5.
- وغالبا الصحيح واحد أو اثنين فقط، chunks طبيعي أقل لأننا نرجع 5: Precision@5.
- يوضح أن الدليل الصحيح ليس موجودا فقط، بل غالبا مرتب جيدا: MAP@5.
- يوضح مكان أول دليل صحيح. الرقم العالي يعني أول إجابة صحيحة تظهر مبكرا: MRR.
- refusal threshold لها لتحديد score تراجع، colorectal guideline أسئلة لا تخص: Out-of-scope.

13. طريقة التشغيل

```
cd "F:\Creativa Hackathon"
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
python ingest.py
python evaluate.py --top-k 5
python query.py "What follow-up is needed after surgery?"
python generate.py --interactive

cd web
npm install
npm run dev
```

ملفات البيئة المطلوبة:

```
Root .env:
GROQ_API_KEY=your_key

web/.env.local:
ANALYSIS_PASSWORD=your_password
```

14. نقاط مهمة لو اتسألتم

- المشروع ليس chatbot طبي عام؛ هو assistant مقيد بإرشادات NICE فقط.
- لو السؤال خارج الدليل، المفروض يرفض بدل ما يخترع.
- الإجابة العربية ممكن تكون ترجمة للنص الداعم، لكن المصدر الأصلي من NICE.
- الـ Vector database مخزنة محليا في `data/vector_store/chroma`.
- embedding model و Chroma و generation client يتم تحميلهم. بعدها web_chat_bridge يحافظ على process شغال لتقليل الوقت.
- أول سؤال قد يأخذ وقتا لأن
- صفحة Analysis خاصة وليست زرا عاما للمستخدم النهائي.
- config.TOP_K داخل صفحة التحليل لا تغير k ثابت في الإنتاج، وتجربة k=5.
- الحالي لا يعني ضمان 100% لأي سؤال في العالم؛ يعني أن النظام يغطي أسئلة الاختبار الحالية داخل النطاق test set على 100% Found Rate.